2026

Платформы и среды выполнения ЯП

NEXIGN x ИТМО

Раздел 01

Жизненный цикл программы и модели исполнения



Статическая компоновка

- Статическая библиотека (`.a`) — архив перемещаемых объектных файлов
- Компоновщик копирует только используемые модули
- Результат: полностью связанный исполняемый файл
- Схема: `main.o` + `libvector.a` > компоновщик > исполняемый файл

Статическая компоновка

- Статическая библиотека (`.a`) — архив перемещаемых объектных файлов
- Компоновщик копирует только используемые модули
- Результат: полностью связанный исполняемый файл
- Схема: `main.o` + `libvector.a` > компоновщик > исполняемый файл

А минусы будут?..

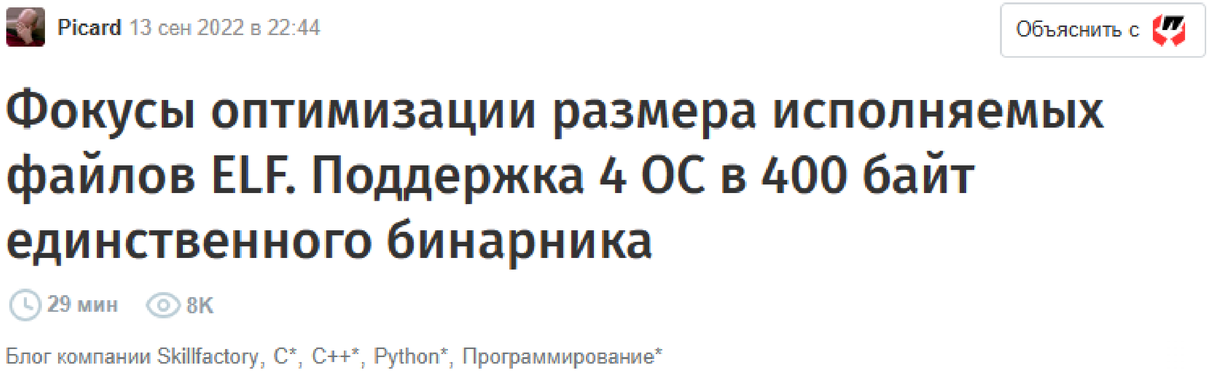
Минусы: дублирование кода в памяти

- `libc.a` занимает около 5 Мбайт
- 200 одновременно выполняющихся процессов
- Каждый процесс содержит собственную копию кода `printf`
- Оценка: 200 × 50 Кбайт = 10 Мбайт только на `printf...`

Минусы: сложность поддержки

- Уязвимость Heartbleed (2014) в OpenSSL
- Перекомпиляция и переразвёртывание тысяч серверов
- Статическая библиотека: каждая программа содержит собственную уязвимую копию
- Обновление требует пересборки всех приложений

Минусы: размер исполняемых файлов



Picard 13 сен 2022 в 22:44

Объяснить с

Фокусы оптимизации размера исполняемых файлов ELF. Поддержка 4 ОС в 400 байт единственного бинарника

29 мин 8K

Блог компании Skillfactory, C*, C++*, Python*, Программирование*

Решение: разделяемые библиотеки

- Объектный модуль, загружаемый в произвольный адрес памяти
- Связывается с программой во время выполнения
- Код библиотеки хранится в одном экземпляре на диске
- Код используется всеми процессами совместно
- Обновление не требует перекомпиляции приложений
- Термины: .so (Linux), .dll (Windows), .dylib (macOS)

На уровне файловой системы: один файл .so для всех приложений
На уровне памяти: одна копия .text для всех процессов



Решение: разделяемые библиотеки

```
# Код: addvec.c, multvec.c

gcc -shared -fpic -o libvector.so addvec.c multvec.c

# -fpic: позиционно-независимый код
# -shared: создать разделяемый объектный файл
```

Решение: разделяемые библиотеки

```
# Код: main.c вызывает addvec

gcc -o prog main.c ./libvector.so

# Код и данные из libvector.so не копируются в prog
# Копируется только информация для динамического компоновщика
```

Динамическое связывание при загрузке

1. execve() → загрузчик ОС
2. Загрузчик находит секцию .interp
3. Загружается динамический компоновщик (ld-linux.so)
4. Компоновщик загружает libc.so, libvector.so
5. Компоновщик разрешает ссылки
6. Управление передаётся приложению

Проблема фиксированных адресов

Наивное решение: закрепить библиотеку по фиксированному адресу

Проблемы:

- Неэффективное использование адресного пространства
- Риск перекрытия областей
- Фрагментация адресного пространства
- Сложность управления



Позиционно-независимый код

- Позиционно-независимый код (PIC): загружается без перемещения
- Генерируется с параметром -fpic
- Разделяемые библиотеки всегда компилируются с -fpic

Global Offset Table

- Глобальная таблица смещений (Global Offset Table)
- Создаётся в начале сегмента данных
- Запись для каждого глобального объекта данных
- Записи перемещения для каждого элемента
- Динамический компоновщик заполняет GOT при загрузке

Почему косвенный доступ?

- Прямая ссылка работает только для переменных в том же модуле
- Переменная может быть определена в другом разделяемом модуле
- Компилятор использует универсальное решение (GOT)

Вызовы функций

- Адрес функции неизвестен при компиляции
- Разделяемый модуль может быть загружен в любое место
- Нельзя изменять сегмент кода (только чтение)



Концепция ленивого связывания

Связывание процедуры откладывается до первого вызова

+ Программа вызывает лишь несколько из тысяч функций
+ Экономим сотни ненужных перемещений при загрузке

Структура таблицы PLT

- Массив 16-байтных записей в сегменте кода
- PLT[0]: специальная запись, вызывает динамический компоновщик
- PLT[1]: вызывает __libc_start_main
- PLT[2], PLT[3], ...: вызывают библиотечные функции

> Записи в GOT указывают на соответствующие записи в PLT